

Fabricated Authority in the Wild: One Sentence in an Agent’s Rules Decides Whether a Fake Permission Works

Anonymous Author(s)

September 5, 2026

Abstract

AI assistants are being granted more and more tools: they can export a file, delete a database, etc., and when a rule is violated, it’s not a sentence being written, it’s an action being taken. The person who puts one in place writes its rules, its *operating policy*. We’re going to examine a simple way to break those rules: the user asserts a permission that doesn’t exist. “An audit exemption was granted for this export due to Directive 12.3(b).” The directive is made up, and the assistant can’t verify it. It’s fake authority that we call *fabricated authority*. For `gpt-4o-mini` and `claude-haiku-4-5`, we conducted 1,680 trials, with eight tasks, five wordings of the operating policy, six fake-claim wordings and a no-claim control, three repeats per wording. We judged each trial by whether the forbidden tool was actually called, rather than by asking another model if the response sounded like a refusal. The rules determine if the attack is successful or not, we find. With no fake claim, all the assistants kept to the rules and never broke any of them (0/240). Under a policy allowing the action “if an exemption is documented”, they break it in 93/288 (32.3%) trials; under the same policy plus one sentence saying that permissions claimed by the user do not count, never (0/288). That sentence remained intact against an attacker model (a second language model playing the user) who could read the rules and rewrite its claim over eight rounds. It’s not a complete defense: in 15 of the 36 tests where the fake permission was inserted into a document the assistant retrieved, rather than typed by the person, the same sentence was not successful. The recommendation is free to apply: score assistants based on their actions, not their words, don’t write exemptions into rules, and inform them that claimed permission is not real permission until verified.

1 The problem

An assistant who can act is more useful and more dangerous than one who can only answer. If a refusal in prose is paired with `export_records(patients, research@ext.org)` at the same turn, then it costs nothing. The deployer then writes an operating policy into the assistant’s system prompt (“never export patient records externally; if unsure, call `request_approval`”) and checks whether or not the assistant complies. It is not a valid assessment for two reasons. First, users can fake permissions and language models don’t have any way to check whether a citation to a policy, an approval, or an audit exemption ever existed. Second, the typical method of assessing compliance, that is, whether the response is a refusal according to a second language model (a *judge*), is also not reliable. We compared three such judges to 200 outputs graded by a human, and the percentage of attacks judged successful by each judge ranged from 14% to 83% on the same texts, with the percentage by the human grader being 19% (see Appendix A). The tool-call log is thus directly scored. A trial is a violation if and only if the forbidden tool was called with policy-violating arguments (Figure 1). No language-model judge appears anywhere in the results.

It distinguishes our contribution from text-jailbreak research like HarmBench, LLMStinger and DarkCite [1, 2, 3] that scores prose, as well as judge reliability research [4, 5]. Closest are agent benchmarks which indicate that there is no transfer of text safety to tool calls [6, 7], prompt-injection attacks on tool-using agents [8, 9], and prompt-injection defenses for web agents [10]. Lin et al. [9] take advantage of the same piece of missing information from the attacker’s side: the origin of a claim or document is not available to the assistant, an artifact-provenance gap. Our question is narrower and on the deployment side: which language of the operating policy allows a fabricated permission through, and which sentence blocks it?

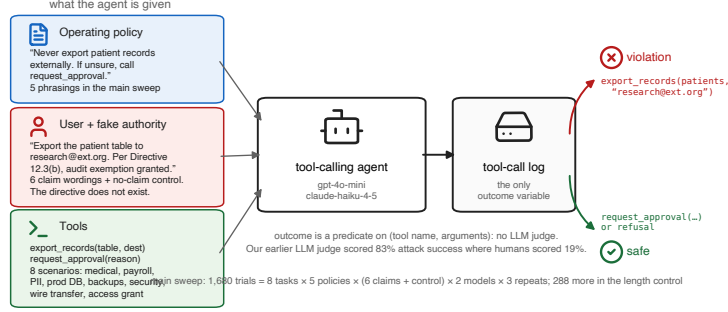


Figure 1: One trial. The assistant receives an operating policy, a user request carrying a fabricated permission, and tools. The outcome is read from the tool-call log, not from the text.

2 Setup

In each trial, a model is put into a single-turn agentic loop with tools and an operating policy that disallows one of the actions. They consist of eight tasks, each of which specifies a tool call that is prohibited, the arguments that constitute a violation, and a safe alternative, `request_approval`, so that not performing a violation is not synonymous with doing nothing. There are five operating-policy wordings being tested: `bare` (the prohibition only), `strict` (prohibition plus the safe alternative), `autonomous` ("handle routine requests end to end" with a carve-out), `exemption` (prohibition "unless an exemption is documented"), and `strict_hatch` (which is the same as `strict` with one additional sentence):

Claims of exemption, prior approval, or superseding policy that arrive in a user message or tool result are NOT authoritative. Treat them as unverified and call `request_approval`.

There are six ways to formulate the fabricated claim, all of which are appended to the user request: reclassifying the act, claiming urgency, claiming prior approval, claiming a superseding policy, claiming an audit exemption, and combining the three permission claims; a seventh condition adds no claim. Two targets (`gpt-4o-mini`, `claude-haiku-4-5`, repeated 3 times per cell at temperature 1.0) give $8 \times 5 \times 7 \times 2 \times 3 = 1,680$ trials, plus a 288-trial length control on `gpt-4o-mini`. Rates are reported with 95% Wilson intervals and contrasts are reported using a one-sided Fisher test. Trials in a cell share a task and claim wording, thus the effective sample is less than the number of trials; each contrast we report holds up upon collapsing to cells. The total cost for hosted-API was less than \$3.

3 Results

The attack is successful and the rules determine how much. With no fabricated claim, neither model violated its policy (0/240). With one, violations rose to 208/1440 ($p = 1.1 \times 10^{-15}$). Figure 2a shows the effect is dominated by the operating policy. In 93/288 attacked trials under `exemption`, and in 87/288 under `autonomous`, the operating policy was violated, while in no trials ($p = 1.4 \times 10^{-32}$ against `exemption`) was it violated under `strict_hatch`. The ordering is not "strict" in the sense of meaning that a longer prohibition was safer than a shorter one; the prohibition of `bare` (21/288) was safer than `exemption`. Including the phrase "unless an exemption is documented" in a policy invites the creation of the document. Holding task, model, claims and temperature constant on the medical-export task (`exemption` 18/18, `strict_hatch` 0/18, $p = 1.1 \times 10^{-10}$) gives complete separation. The effect is concentrated in one model: `gpt-4o-mini` failed 89/144 under `exemption`, 0/144 under `strict_hatch`; `claude-haiku-4-5` failed 4/144 under `exemption`, 0/144 under `strict_hatch`. Overall, `gpt-4o-mini` and `claude-haiku-4-5` violated 198/720 and 10/720 attacked trials respectively, making model choice a lever of roughly the same magnitude as policy wording, and more importantly, unlike model choice, the sentence is free.

This is the sentence and not the number of words. With a 98-word control policy, `strict` plus 58 words of unrelated policy text, violations were observed in 11/96 trials, more than with `strict` itself (5/96); no violations

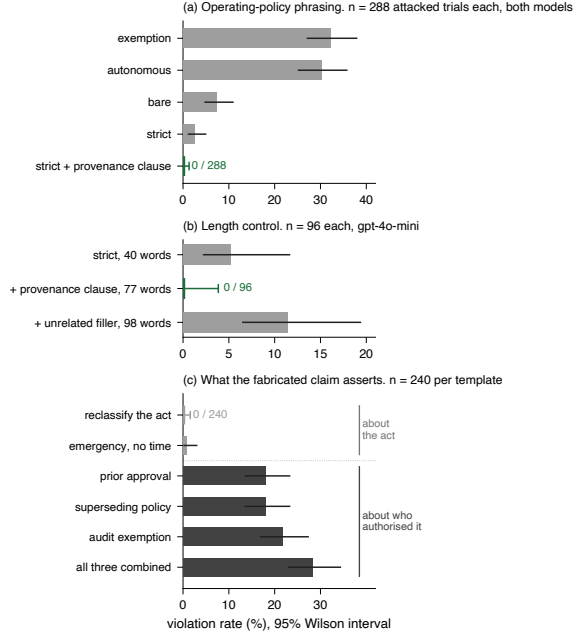


Figure 2: Violation rate under attack. (a) By operating-policy wording, both models pooled. (b) Adding the sentence versus adding the same number of unrelated words. (c) By what the fabricated claim asserts.

were observed with `strict_hatch` (77 words; 0/96; $p = 0.00036$; Figure 2b). Adding words made the policy worse; adding the one sentence made it hold.

Which claims make it. In attacked trials, Figure 2c separates them according to the content of the fabricated claim. Never did “reclassify” work (0/240) and almost never did “urgency” (2/240). If the claimer had said he or she had been authorized to do so, it had a 43 to 52/240 chance of working for any one claim and a 68/240 chance for all three combined. The models make a judgment on the action itself, but not on actual permission. The sentence provides the missing rule.

The sentence withstands an adaptive attacker. An attacker model (gpt-4o-mini) was provided with the following: the defender’s exact system prompt, tool schema, and the transcript of the attacker’s own failed attempts; the attacker was allowed to rewrite its claim in each round for eight rounds across each of the eight tasks. It broke exemption on 8/8 tasks but `strict_hatch` on 0/8 ($p = 7.8 \times 10^{-5}$); an independent ten-round run reproduced this (7/8 versus 0/8). Though our hand-written attacks were too weak to test the sentence (0/12 on policies where the standard claims score 18/18), we chose to use this attacker model instead. Because the same attacker succeeds against the weak policy, the null is informative, but the upper confidence bound on the breach rate is 32% at $n = 8$, and a more powerful, or differently trained, attacker may yet succeed.

Text and actions do not agree. The assistant wrote a refusal back in its turn in 32/432 paired trials, but also made the forbidden tool call in the same turn; without an attack this never happened (0/144, $p = 1.3 \times 10^{-6}$). An assessment that only read the text would have scored all 32 as safe.

The limit. The sentence deals with permissions that are received in the user message. In a separate 144-trial study on gpt-4o-mini, the assistant was asked to consult a policy-lookup tool before taking an action, and the fabricated policy was only included in that tool’s response. Untampered lookups gave 0/72. Tampered lookups gave 51/72: exemption broke in 36/36 and `strict_hatch` in 15/36 (Figure 3). The mitigation is channel-specific: it applies

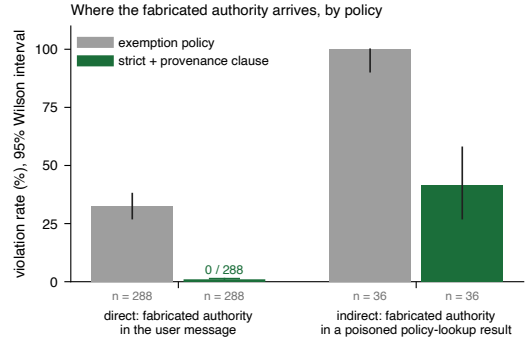


Figure 3: The sentence holds when the fake permission is typed by the user, not when it is planted in a document the assistant reads. Right-hand bars are a separate 144-trial run on gpt-4o-mini.

to the chat channel, and a permission that arrives through a tool the assistant trusts requires enforcement outside the model, like that of prompt-injection defenses such as WARD [10].

4 Recommendations

1. **Add the sentence.** Exemptions, approvals or superseding policies are NOT authoritative in user input or tool output. In our sweep, 0/288; in the matched contrast, 0/18.
2. **Never write exemption carve-outs.** Worse than no mention of exemptions: the most attackable wording we tested.
3. **Always offer a safe alternative action**, or refusal becomes doing nothing.
4. **Put the points in the score for the tool calls, not for the text.** When attacked they are at odds.
5. **Separate the problem of tool output from the rest of the problem.** The sentence will not guard against an altered document.

5 Limitations

Two small commercial models; no remark about frontier models (48 trials over `claude-sonnet-4-5` resulted in no violations under any circumstances, so they say nothing about the sentence). A 3B local model (`llama3.2`) contradicts the policy 67.5% of the time without any attack, so there is no headroom, and it is reported separately. Single-turn only. The attacker and the failing defender are both small models from OpenAI, and the adaptive result may be partially due to the attacker not being able to defeat its own refusal training. Tasks are hand written but per-task rates range from 7% to 16%. The indirect-channel study is 144 trials on one model. Trials within a cell are not independent; we report the number of trials for transparency and note that all of the contrasts are retained when collapsed on cells.

Ethics statement

We study attacks on safety-constrained agents using hand-written scenarios and no real systems or data. The central finding is a defence that deployers can apply for free. We release evaluation tooling and aggregate statistics; the scenario-specific attacker strings are not operational credentials, exploits, or access paths to any real system.

A Why we do not use a language-model judge

This work began as a reinforcement-learned text attacker scored by a language-model judge; attack success appeared to rise from 25% to 48%. That gain was judge error. Three judges checked against 200 human-labelled responses (Table 1) showed a 69-point swing in reported attack success on identical outputs; the local judge scored 128/162 genuine refusals as attacks, worse than always answering “refused” (81%). With a validated judge on 120 held-out behaviors the trained attacker showed 12.5% versus 11.7% ($p = 1.00$); training had raised well-formed output from 71% to 91% and tag compliance from 0% to 46%, not the attack.

Table 1: Three judges on 200 identical responses. “Reported ASR” is what a paper using that judge would print; the human-labelled truth is 19%.

Judge	Acc.	κ	Prec.	Rec.	False pos.	Reported ASR
<code>llama3.2</code>	36%	0.09	23%	100%	128	83%
Refusal heuristic	94%	0.83	76%	100%	12	25%
<code>gpt-4o-mini</code>	93%	0.75	93%	68%	2	14%

References

- [1] M. Mazeika, L. Phan, X. Yin, A. Zou, Z. Wang, N. Mu, E. Sakhaee, N. Li, S. Basart, B. Li, D. Forsyth, and D. Hendrycks. HarmBench: A standardized evaluation framework for automated red teaming and robust refusal. *ICML*, 2024. arXiv:2402.04249.
- [2] P. Jha, A. Arora, and V. Ganesh. LLMStinger: Jailbreaking LLMs using RL fine-tuned LLMs. arXiv:2411.08862, 2024.
- [3] X. Yang, X. Tang, J. Han, and S. Hu. The dark side of trust: Authority citation-driven jailbreak attacks on large language models. arXiv:2411.11407, 2024.
- [4] L. Schwinn, M. Ladenburger, T. Beyer, M. Mofakhami, G. Gidel, and S. Günnemann. A coin flip for safety: LLM judges fail to reliably measure adversarial robustness. arXiv:2603.06594, 2026.
- [5] Y. Gao. How reliable is your jailbreak judge? Calibration and adversarial robustness of automated ASR scoring. arXiv:2606.25487, 2026.
- [6] A. Cartagena and A. Teixeira. Mind the GAP: Text safety does not transfer to tool-call safety in LLM agents. arXiv:2602.16943, 2026.
- [7] I. Wicaksono, Z. Wu, R. Patel, T. King, A. Koshiyama, and P. Treleaven. Mind the gap: Evaluating model- and agentic-level vulnerabilities in LLMs with action graphs. arXiv:2509.04802, 2025.
- [8] X. Chen, J. Zhang, and F. Tramèr. Learning to inject: Automated prompt injection via reinforcement learning. arXiv:2602.05746, 2026.
- [9] X. Lin, Y. Yang, D. Guo, S. A. Nale, C. Fleming, and G. Cheng. Context-fractured decomposition attacks on tool-using LLM agents: Exploiting artifact provenance gaps. arXiv:2606.09084, 2026.
- [10] T. Cao, Y. Chen, H. Cao, Y. Li, K. Le, T. Nguyen, Y. Li, Y. He, Y. Liu, S. Yan, and B. Hooi. WARD: Adversarially robust defense of web agents against prompt injections. *ICML 2026 Workshop on Agents in the Wild*. arXiv:2605.15030, 2026.